Database & SQL Fundamentals

Institute Management System – Essential Topics

1. Database & SQL Basics

What is a Database?

A database is a structured collection of data stored electronically so that it can be easily accessed, managed, and updated.

Examples:

- Student records
- Course details
- Login credentials

In this project:

- Users (email, password, role)
- Students (name, course, mobile)
- Courses (fees, duration)
- Videos (course-wise content)

What is RDBMS?

RDBMS (Relational Database Management System) stores data in tables (rows and columns) and maintains relationships between tables.

Examples:

- `students` table is related to `courses`
- `students.email` is linked to `users.email`

MySQL is an RDBMS.

What is SQL?

SQL (Structured Query Language) is used to:

- Create databases
- Create tables

SUNBEAM

- Insert, update, and delete data
- Retrieve data

MySQL uses SQL syntax.

---

2. Data Types (Used in This Schema)

Data Type	Used For	Explanation
INT	course_id, reg_no, fees	Stores numeric values
VARCHAR	name, email, title	Stores text
DATE	start_date, end_date	Stores date only
DATETIME	added_at	Stores date and time
ENUM	role	Fixed set of values
BLOB	profile_pic	Binary data (image/file)

Example:

```
email VARCHAR(100);
```

---

3. DDL – Data Definition Language

Data Definition Language (DDL) is a set of SQL commands used to define, create, and modify the structure of database objects such as databases and tables.

DDL commands do not work on data; instead, they control how data is stored.

DDL changes are permanent because they modify the database schema.

CREATE DATABASE

Creates a new database

```
CREATE DATABASE IF NOT EXISTS sunbeam_institute_management_db;
```

USE DATABASE

Selects the database to work with

```
USE sunbeam_institute_management_db;
```

CREATE TABLE

Creates a new table with specified columns and data types

```
CREATE TABLE users (
    email VARCHAR(100),
    password VARCHAR(255),
    role ENUM('student','admin')
);
```

Purpose of DDL

- To design the database structure
- To define tables and their columns
- To control how data will be stored in the database

---

4. DML – Data Manipulation Language

Data Manipulation Language (DML) is a set of SQL commands used to add, modify, retrieve, and remove data stored in database tables.

DML works on the data inside tables, not on the structure.

DML operations can be rolled back if transactions are used.

INSERT

Adds new records into a table

```
INSERT INTO users (email, password, role)
VALUES ('abc@gmail.com', 'hashedpwd', 'student');
```

SELECT

Retrieves records from a table

```
SELECT * FROM users;
```

SUNBEAM

UPDATE

Modifies existing records

```
UPDATE users
SET role = 'admin'
WHERE email = 'abc@gmail.com';
```

DELETE

Removes records from a table

```
DELETE FROM users
WHERE email = 'abc@gmail.com';
```

Purpose of DML

- To store new data in tables
- To modify existing data
- To retrieve required data
- To remove unwanted data

---

5. DQL – Query Conditions

Data Query Language (DQL) is a subset of SQL used specifically to retrieve and filter data from database tables.

DQL focuses only on reading data and does not modify the database.

DQL commands are often used with conditions and sorting.

WHERE

Filters records based on conditions

```
SELECT * FROM users
WHERE role = 'student';
```

AND / OR

Combines multiple conditions

```
SELECT * FROM users
WHERE role = 'student' AND email LIKE '%gmail%';
```

LIKE

Searches using patterns

```
SELECT * FROM users
WHERE email LIKE '%@gmail.com';
```

ORDER BY

Sorts the result set

```
SELECT * FROM users
ORDER BY email ASC;
```

Purpose of DQL

- To retrieve required data
- To apply conditions on data
- To sort and filter query results

---

6. Constraints

1. PRIMARY KEY

Theory

- PRIMARY KEY uniquely identifies each record in a table.
- A table can have only one PRIMARY KEY.
- PRIMARY KEY does not allow NULL or duplicate values.

SQL Code

```
CREATE TABLE demo_primary_key (
    reg_no INT PRIMARY KEY,
```

```
    name VARCHAR(50) NOT NULL
);

INSERT INTO demo_primary_key VALUES (1, 'Amit');
INSERT INTO demo_primary_key VALUES (2, 'Sneha');
```

Output

```
Query OK, 1 row affected
Query OK, 1 row affected
```

Duplicate Entry Test

```
INSERT INTO demo_primary_key VALUES (1, 'Rahul');
```

Output

```
ERROR 1062: Duplicate entry '1' for key 'PRIMARY'
```

---

2. AUTO_INCREMENT

Theory

- AUTO_INCREMENT automatically generates sequential numeric values.
- Commonly used with PRIMARY KEY.

SQL Code

```
CREATE TABLE demo_auto_increment (
    reg_no INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL
);

INSERT INTO demo_auto_increment (name) VALUES ('Amit');
INSERT INTO demo_auto_increment (name) VALUES ('Sneha');

SELECT * FROM demo_auto_increment;
```

Output

```
+--------+--------+
| reg_no | name   |
+--------+--------+
| 1      | Amit   |
| 2      | Sneha  |
+--------+--------+
```

---

3. NOT NULL

Theory

- NOT NULL ensures that a column must always have a value.

SQL Code

```sql
CREATE TABLE demo_not_null (
    name VARCHAR(50) NOT NULL
);

INSERT INTO demo_not_null VALUES ('Amit');
INSERT INTO demo_not_null VALUES (NULL);
```

Output

```
ERROR 1048: Column 'name' cannot be null
```

---

4. UNIQUE

Theory

- UNIQUE constraint prevents duplicate values in a column.

SQL Code

```sql
CREATE TABLE demo_unique (
    email VARCHAR(100) NOT NULL UNIQUE
```

```
);

INSERT INTO demo_unique VALUES ('abc@gmail.com');
INSERT INTO demo_unique VALUES ('abc@gmail.com');
```

Output

```
ERROR 1062: Duplicate entry 'abc@gmail.com' for key 'email'
```

---

5. DEFAULT

Theory

- DEFAULT assigns a predefined value when no value is provided.

SQL Code

```
CREATE TABLE demo_default (
    role VARCHAR(20) DEFAULT 'student'
);

INSERT INTO demo_default VALUES ();
SELECT * FROM demo_default;
```

Output

```
+---------+
| role    |
+---------+
| student |
+---------+
```

---

6. ENUM

Theory

- ENUM restricts column values to a fixed set of predefined values.

SQL Code

```
CREATE TABLE demo_enum (
    role ENUM('student','admin')
);

INSERT INTO demo_enum VALUES ('student');
INSERT INTO demo_enum VALUES ('teacher');
```

Output

```
ERROR 1265: Data truncated for column 'role'
```

---

7. Keys & Relationships

FOREIGN KEY

A FOREIGN KEY is a constraint used to link two tables. It ensures referential integrity, meaning:

A value in the child table must exist in the parent table

Invalid or orphan data is not allowed

A FOREIGN KEY always:

Refers to a PRIMARY KEY or UNIQUE key

Exists in the child table

Points to the parent table

Real-World Example

users → parent table

students → child table

A student must have a valid user account

Step 1: Parent Table

SUNBEAM

```
DROP TABLE IF EXISTS users;

CREATE TABLE users (
    email VARCHAR(100) PRIMARY KEY
);
```

Step 2: Child Table with FOREIGN KEY

```
CREATE TABLE students (
    reg_no INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    FOREIGN KEY (email) REFERENCES users(email)
);
```

Step 3: Valid Insert Order

```
INSERT INTO users VALUES ('rahul@gmail.com');

INSERT INTO students (name, email)
VALUES ('Rahul', 'rahul@gmail.com');
```

One-to-Many Relationship

A One-to-Many relationship means:

One record in the parent table

Can be associated with multiple records in the child table

This is implemented using a FOREIGN KEY.

- One course → many students
- One course → many videos

Step 1: Parent Table

```
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
```

```
    course_name VARCHAR(100)
);
```

Step 2: Child Table

```
CREATE TABLE students (
    reg_no INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    course_id INT,
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

Step 3: Insert Data

```
INSERT INTO courses VALUES (1, 'Full Stack Java');

INSERT INTO students (name, course_id)
VALUES
('Amit', 1),
('Sneha', 1),
('Rohit', 1);
```

Step 4: View Relationship

```
SELECT * FROM students;
```

ON DELETE CASCADE

ON DELETE CASCADE is a FOREIGN KEY option that automatically deletes child records when a parent record is deleted.

Without CASCADE:

Parent deletion fails if child records exist

With CASCADE:

Parent deletion succeeds

All related child records are deleted automatically

Real-World Example

Delete a course

All videos related to that course should be deleted automatically

Step 1: Parent Table

```
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100)
);
```

Step 2: Child Table with CASCADE

```
CREATE TABLE videos (
    video_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(100),
    course_id INT,
    FOREIGN KEY (course_id)
    REFERENCES courses(course_id)
    ON DELETE CASCADE
);
```

Step 3: Insert Data

```
INSERT INTO courses VALUES (1, 'Python');

INSERT INTO videos (title, course_id)
VALUES
('Intro to Python', 1),
('Loops in Python', 1);
```

Step 4: Delete Parent Record

```
DELETE FROM courses WHERE course_id = 1;
```

Step 5: Verify Child Records

```
SELECT * FROM videos;
```

All related videos are deleted automatically.

8. Joins

INNER JOIN

An INNER JOIN returns only those records where there is a matching value in both tables.

Rows without a match in either table are excluded

Used when data must exist in both tables

Real-World Scenario

Students are enrolled in courses

Show student name along with course name

Only students who are enrolled should appear

Step 1: Create Tables

```
DROP TABLE IF EXISTS students;
DROP TABLE IF EXISTS courses;

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100)
);

CREATE TABLE students (
    reg_no INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    course_id INT
);
```

Step 2: Insert Data

```
INSERT INTO courses VALUES
(1, 'Full Stack Java'),
```

```
(2, 'Python Data Science');

INSERT INTO students (name, course_id) VALUES
('Amit', 1),
('Sneha', 2),
('Rohit', 1);
```

Step 3: INNER JOIN Query

```
SELECT s.name, c.course_name
FROM students s
INNER JOIN courses c
ON s.course_id = c.course_id;
```

LEFT JOIN

A LEFT JOIN returns:

All records from the left table

Matching records from the right table

NULL where no match exists

Used when left table data is mandatory, but right table data is optional.

Real-World Scenario

Courses may or may not have videos

Show all courses, even if videos are not uploaded

Step 1: Create Tables

```
DROP TABLE IF EXISTS videos;
DROP TABLE IF EXISTS courses;

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100)
);

CREATE TABLE videos (
    video_id INT AUTO_INCREMENT PRIMARY KEY,
```

SUNBEAM

```
    title VARCHAR(100),
    course_id INT
);
```

Step 2: Insert Data

```
INSERT INTO courses VALUES
(1, 'Java'),
(2, 'Python'),
(3, 'React');

INSERT INTO videos (title, course_id) VALUES
('Java Basics', 1),
('Advanced Java', 1),
('Python Basics', 2);
```

Step 3: LEFT JOIN Query

```
SELECT c.course_name, v.title
FROM courses c
LEFT JOIN videos v
ON c.course_id = v.course_id;
```

Final Summary

Topic	Purpose
Database Basics	Understand DB & SQL
Data Types	Correct storage
DDL	Create structure
DML	Modify data
DQL	Filter data
Constraints	Data integrity
Relationships	Connect tables
Joins	Real-world queries